Machine Learning in Materials Chemistry Trinity Term 2026
Assignment 1: Gaussian Progress Regression

Candidate no. 1104809

Introduction

In many scientific fields, properties observed on a small set of examples need to be generalised to a wider domain. In materials chemistry, this may involve measuring the energy of a few molecules, or computing it using expensive Density Functional Theory (DFT) methods, and then generalising these findings for the prediciton of properties of other molecules.

In general, the function linking a domain object (e. g. the representation of a molecule) to a property of interest can be non-linear, high-dimensional and difficult to evaluate. There exist different approaches to learn these functions from empirical observations, called training data $\mathcal{D}$:

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^{N} \tag{1}$$

One such data-driven method is gaussian process regression (GPR) which assumes the unknown function to be a gaussian process. This means that for any finite set of input points, the corresponding latent function values are jointly gaussian distributed for some m, k:

$$f \sim \mathcal{GP}(m, k) \tag{2}$$

Observations in the training data might additionally contain noise or approximation errors which is commonly simply modeled as gaussian noise itself:

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2). \tag{3}$$

As discussed in lecture 2 and described in [2], the prediction $\tilde{y}$ at a specific point $\mathbf{x}$ in the domain is then given by

$$\tilde{y}(\mathbf{x}) = \sum_{m=1}^{M} c_m k(\mathbf{x}, \mathbf{x}_m) \tag{4}$$

where c_m are empirically learned coefficients and $k(\mathbf{x}, \mathbf{x}_m)$ is the kernel function, dependent on the similarity of the training inputs $\mathbf{x}_m$ and $\mathbf{x}$ in the domain. In full GPR, which is the focus of the remainder of this project, $M = N$. Sparse GPR, in contrast, can be more computationally efficient, but that might come at the cost of accuracy.

Optimising the choice of coefficients c_m leads to the following loss function which needs to be minimised (under the assumption that all data points have the same expected noise):

$$\mathcal{L} = \sum_{n=1}^{N} [y_n - \tilde{y}(\mathbf{x}_n)]^2 + \sigma^2 \sum_{m,m'=1}^{M} c_m k(\mathbf{x}_m, \mathbf{x}_{m'}) c_{m'} \tag{5}$$

and the solution

$$\mathbf{c} = (\mathbf{K}_{NN} + \sigma^2 \mathbf{I})^{-1} \mathbf{y} \tag{6}$$

where $\mathbf{K}_{NN}$ is the covariance matrix of the training input points with themselves, so $(\mathbf{K}_{NN})_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

Predictions can then be made using Equation 4 leading to:

$$\tilde{y}(\mathbf{x}) = \mathbf{c}^\top \mathbf{k}(\mathbf{x}), \tag{7}$$

This task concerns itself with applying this method to simple 1D functions.

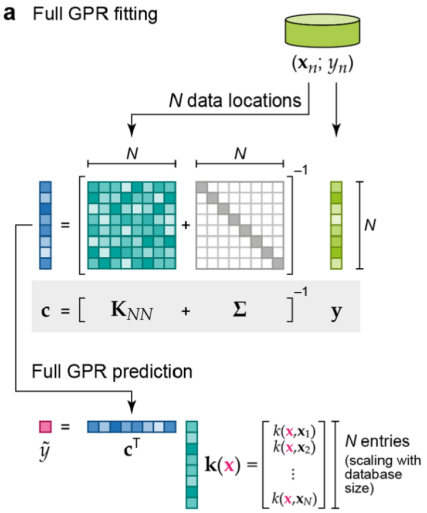


Figure 1: Flowchart illustrating training- and inference process in GPR. The training data (see Eqaution 1) is used to calculate c from Equation 6, involving the matrices $\mathbf{K}_{NN}$ and a diagonal matrix consisting of σ^2 on the diagonal and the training dataset. These coefficients and the covariance of function values (determined by their locations in the domain and calculated through the kernel function) are then used to make a prediction $\tilde{y}$. Figure taken from [2].

Question 1a)

The code was written such that the exploration of various different combinations of kernel functions and 1D functions to be approximated is easily possible, as well as variations in noise level and training set size and hyperparameters that could be optimised using a hyperparameter search. The latter three aspects are beyond the scope of this project except for a (rough) hyperparameter search for the length scale parameter.

Gaussian noise was added to the following example target functions:

$$f_{\sin}(x) = \sin(2x + 3) - 1 \tag{8}$$

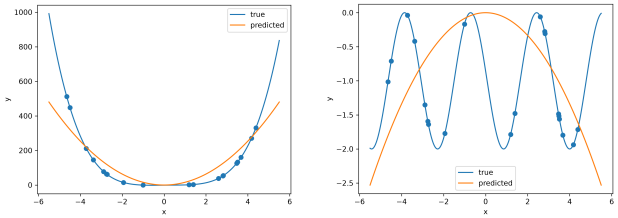
$$f_{\text{poly},2}(x) = 3x^2 + 4x - 5 \tag{9}$$

$$f_{\text{poly},4}(x) = x^4 - 0.5x^3 + x - 1 \tag{10}$$

$$f_{\exp}(x) = 0.1e^{-0.5x} \tag{11}$$

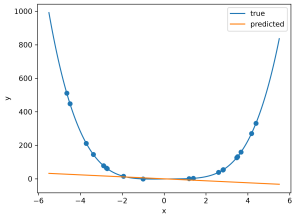
The implementation of the three investigated kernel functions (see Equation 12) was taken from the openly available repository `pykernels` ([1]), with a few adjustments needed for a newer python version.

$$\begin{aligned} k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') &= \sigma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \\ k_{\text{poly}}(\mathbf{x}, \mathbf{x}') &= (\mathbf{x}^\top \mathbf{x}' + c)^d \\ k_{\text{lin}}(\mathbf{x}, \mathbf{x}') &= \mathbf{x}^\top \mathbf{x}' \end{aligned} \tag{12}$$

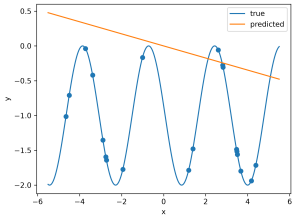


(a) polynomial function, polynomial kernel, $d = 4$

(b) sine function, polynomial kernel, $d = 4$



(c) polynomial function, linear kernel



(d) sine function, linear kernel

Figure 2: Comparison of polynomial and linear kernels for polynomial and sine target functions (see Equations 10 and 8). The polynomial kernel of degree 4 is suitable to approximate the polynomial function of degree 4 in the training domain $[-5, 5]$, but not the periodic sine function, whilst the linear kernel is not suitable for either.

Results

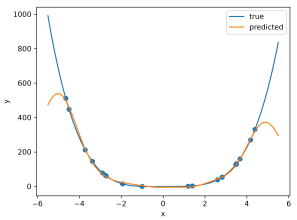
The polynomial and linear kernels were only suitable to approximate the polynomial example functions and only up to the maximum degree of the kernel itself (see Figure 2). This was to be expected as the space that polynomials of a fixed degree d spans is only that of functions that are themselves polynomials of max. degree d (and because of Equation 7, GPR models only such linear combinations of basis functions).

The RBF kernel on the other hand is suitable for the approximation of a variety of functions using GPR, see Figure 3

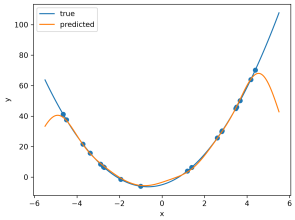
The effect of the length scale parameter l in the RBF kernel is especially interesting to investigate because it determines how far the influence of one training input point reaches. A too small value for l can lead to overfitting (because the value of the training data point is only taken into account for the prediciton in a very small region around itself), but a value that is too large can smoothen out the curve too much as can be seen in Figure 4. Figure 5 depicts the root mean squared error (RMSE), evaluated at 100 randomly chosen test points (and normalised to make comparable between functions) and underlines the importance of choosing an appropriate length scale.

Question 1b)

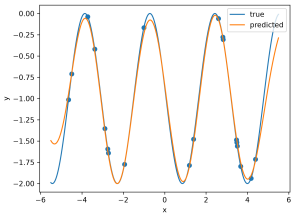
Additionally to using sample function values themselves to approximate the unknown function, one can also use derivative values, because deriving a function is a linear operator and Gaussian processes are closed under linear operators - such that the resulting vector of both the function values and/or derivative values is still a Gaussian process (if f was one in the first place, as assumed through Equation 2. This is particularly useful in sciences such as chemistry and materials science, where some physically relevant quantities are naturally given as derivatives of other properties. For example, in atomistic simulations, the



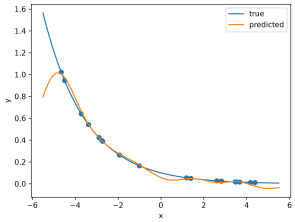
(a) polynomial function (Eq. 10)



(b) polynomial function (Eq. 9)



(c) sine function (Eq. 8)



(d) exponential function (Eq. 11)

Figure 3: Comparison of different target functions using the RBF kernel in GPR and a length scale of 1. In the training domain, the model approximates the given functions well, although it generalises worse outside of the training domain than for example the polynomial kernel on a polynomial function of a similar degree (as is expected due to different assumptions about the underlying function). Because of the regularising term, the predicted function does not necessarily match up with the target function at the training points (trade-off between accuracy and smoothness, also to avoid possible overfitting to noise).

potential energy E is a function of the atomic positions $(\mathbf{R})$, and the forces acting on the atoms are given by the negative gradient of the energy,

$$\mathbf{F} = -\nabla_{\mathbf{R}} E(\mathbf{R}).$$

Thus, force observations correspond to derivative observations of the underlying energy function.

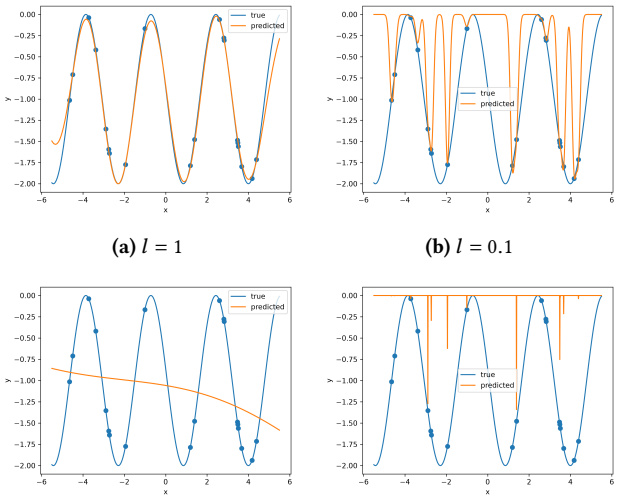
One data point then looks like this (note that N and M lose their definitions from before and are simply counting variables):

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \\ d_1 \\ \vdots \\ d_M \end{bmatrix} = \begin{bmatrix} f(x_1) + \epsilon_1 \\ \vdots \\ f(x_N) + \epsilon_N \\ f'(z_1) + \eta_1 \\ \vdots \\ f'(z_M) + \eta_M \end{bmatrix} \tag{13}$$

For covariances of function and derivative values, the following relationship holds:

$$\begin{aligned} \mathrm{Cov}(f(x), f(x')) &= k(x, x'), \\ \mathrm{Cov}(f(x), f'(x')) &= \frac{\partial}{\partial x'} k(x, x'), \\ \mathrm{Cov}(f'(x), f(x')) &= \frac{\partial}{\partial x} k(x, x'), \\ \mathrm{Cov}(f'(x), f'(x')) &= \frac{\partial^2}{\partial x \, \partial x'} k(x, x'). \end{aligned} \tag{14}$$

This means in order to calculate the covariance between two function values (as previously done by calculating $k(\mathbf{x}_i, \mathbf{x}_j)$), one



(a) $l = 1$ **(b)** $l = 0.1$

(c) $l = 10$ **(d)** $l = 0.001$

Figure 4: Comparison of different values for the length scale parameter l when using GPR with a RBF kernel to model a sine function 8. Too small values (here depicted: $l = 0.1$ and $l = 0,001$ lead to overfitting, whilst the basis functions with a large value like $l = 10$ cannot accurately model the smaller oscillations of the sine function anymore.

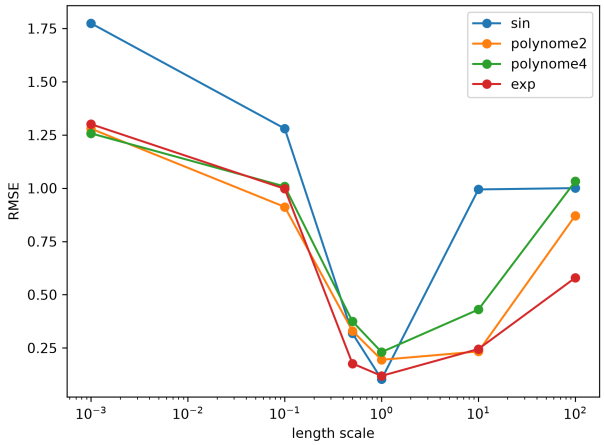


Figure 5: Normalised RMSE over length scale when using GPR with a RBF kernel to model different target functions. In this case, a length scale parameter $l = 1$ achieves the best results.

only needs to differentiate the kernel - which is straightforward in the case of the RBF kernel:

$$\begin{aligned}\frac{\partial}{\partial x'}k(x,x') &= k(x,x')\frac{x-x'}{\ell^2}\\ \frac{\partial}{\partial x}k(x,x') &= k(x,x')\frac{x'-x}{\ell^2}\\ \frac{\partial^2}{\partial x\,\partial x'}k(x,x') &= k(x,x')\left(\frac{1}{\ell^2}-\frac{(x-x')^2}{\ell^4}\right)\end{aligned} \tag{15}$$

Putting these findings together, the covariance matrix becomes:

$$\mathbf{K}_{\text{obs}} = \begin{bmatrix}\mathbf{K}_{ff} & \mathbf{K}_{fd}\\ \mathbf{K}_{df} & \mathbf{K}_{dd}\end{bmatrix} \tag{16}$$

$$\begin{aligned}(\mathbf{K}_{ff})_{ij} &= k(x_i,x_j),\\ (\mathbf{K}_{fd})_{ij} &= \mathrm{Cov}\big(f(x_i),f'(z_j)\big) = \frac{\partial}{\partial z_j}k(x_i,z_j),\\ (\mathbf{K}_{df})_{ij} &= \mathrm{Cov}\big(f'(z_i),f(x_j)\big) = \frac{\partial}{\partial z_i}k(z_i,x_j),\\ (\mathbf{K}_{dd})_{ij} &= \mathrm{Cov}\big(f'(z_i),f'(z_j)\big) = \frac{\partial^2}{\partial z_i\,\partial z_j}k(z_i,z_j).\end{aligned} \tag{17}$$

Results

As can be seen in Figure 6, learning from both the function and derivative values seems to pose a good middle ground in this specific case: Learning purely from derivative values has the issue that a general offset of the function cannot be learned (this corresponds to the missing knowledge of an integration constant) (see the incorrect offset in Figure 6b), but learning from only function values can be a bad estimate for the dynamics in some regions, e. g. in $x \in [1.5, 3.8]$ in Figure 6a.

Conclusion

In this project, Gaussian process regression was applied to simple one-dimensional functions to investigate the influence of kernel choice, the RBF length scale and derivative observations. The results show that the kernel strongly affects the model performance: linear and polynomial kernels work well only for compatible target functions, while the RBF kernel provides more flexibility for different functions.

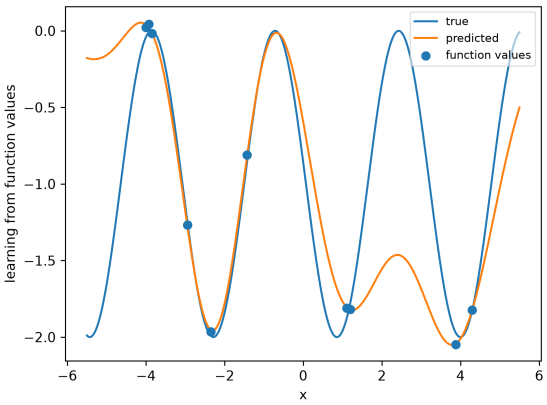
The length scale of the RBF kernel was found to be an important hyperparameter. Very small values can lead to overfitting, whereas very large values oversmooth the prediction. An intermediate length scale therefore gave the best results in the investigated examples.

Including derivative observations extends GPR in a natural way and can improve the description of local function behaviour. However, derivative values alone cannot determine the absolute offset of the function, so combining function and derivative observations provides a useful compromise. Overall, the results demonstrate that GPR is a flexible method, especially when suitable kernels, hyperparameters and physically meaningful derivative information are used.

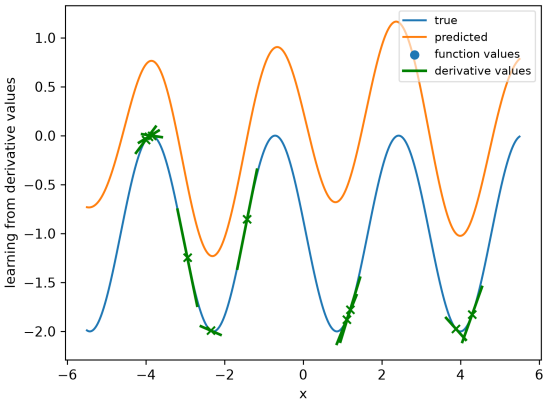
References

[1] Wojciech M. Czarnecki and Katarzyna Janocha. [n. d.]. *pykernels*. https://github.com/gmum/pykernels GitHub repository.

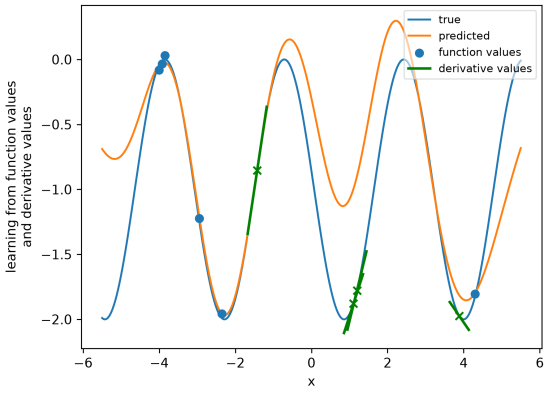
[2] Volker L. Deringer, Albert P. Bartók, Noam Bernstein, David M. Wilkins, Michele Ceriotti, and Gábor Csányi. 2021. Gaussian Process Regression for Materials and Molecules. *Chemical Reviews* 121, 16 (2021), 10073–10141. arXiv:https://doi.org/10.1021/acs.chemrev.1c00022 doi:10.1021/acs.chemrev.1c00022 PMID: 34398616.



(a) polynomial function 10



(b) polynomial function 9



(c) sine function 8

Figure 6: Comparison of GPR predicitons using only function values (6a) (as done in Question 1a), only derivative values (6b) or both (6c). The same number of total training points was used ($N = 10$) in all three approximations. The green tangents denote the usage of the derivative value at the marked point, whilst the blue points corrsepond to the usage of a function value.